

EE243 Project-1 Report

Boran Aybak Kılıç-2022401207

Objective of the Project and Overview of Methodology:

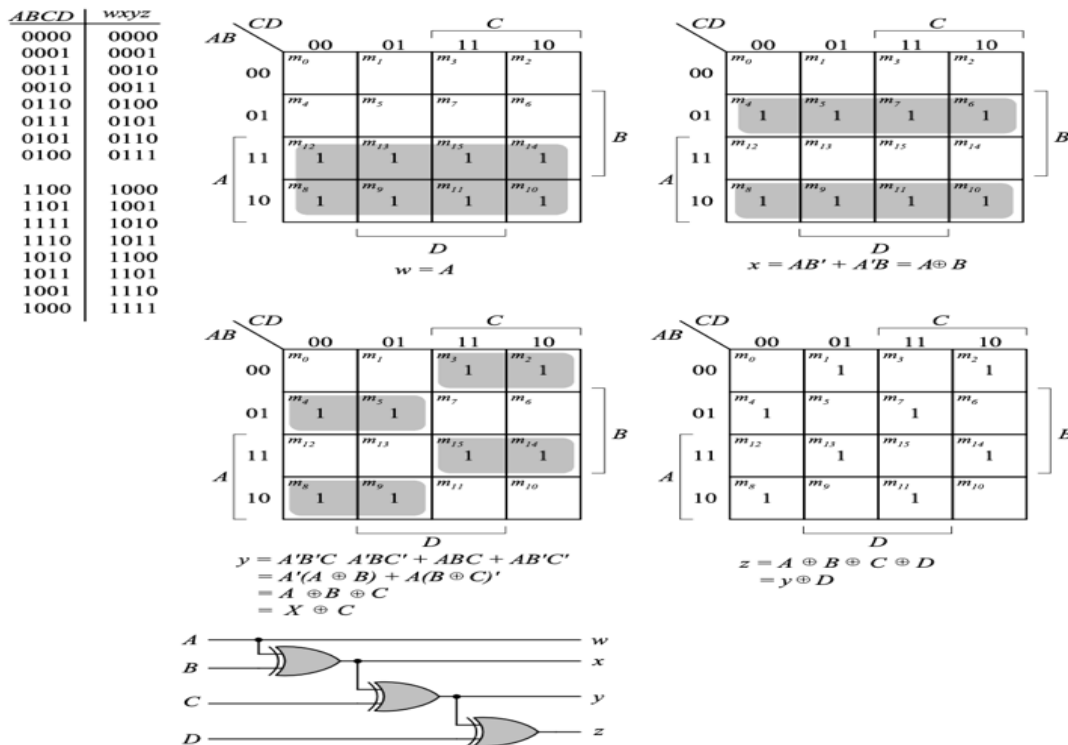
Goal of this project is to design a circuit that converts a 6-bit number in Gray code format to 8-bit BCD (binary coded decimal) format and then display these two BCD numbers with two 7-segment displays. Latter conversion is rather simple as we can draw 4 variable Karnaugh maps representing 4 bits in BCD and repeat this process for all 7 segments. The more difficult part is converting Gray code to BCD. Since binary is more familiar to us and easier to work with, I am going to first convert Gray code to binary (which is very simple) and then convert binary to BCD. The most difficult part of this project was figuring out how to convert a binary to BCD and that is what my main focus will be on this report. Throughout this project I have used a supplementary application called “Digital Logic Sim 2” [1] in which you can easily design and manipulate circuits starting from only AND and NOT gates and build your way up.

Part 1: Converting Gray Code to Binary

By its composition Gray code representation changes by only 1 bit between subsequent numbers. With this property in mind, we need to design a circuit that should respond in some manner if two subsequent bits of the Gray code is different. We can utilize XOR gates for this purpose. A functional circuit can easily be discovered using K-maps.

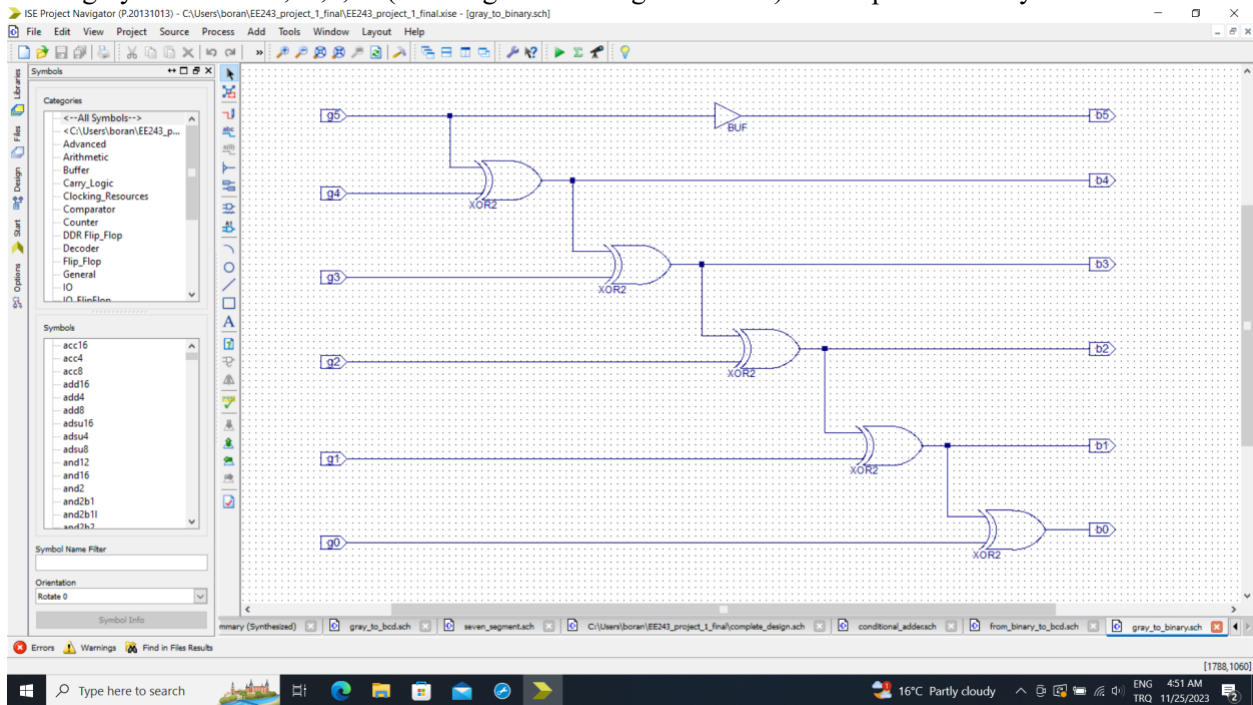
4.7

(a)



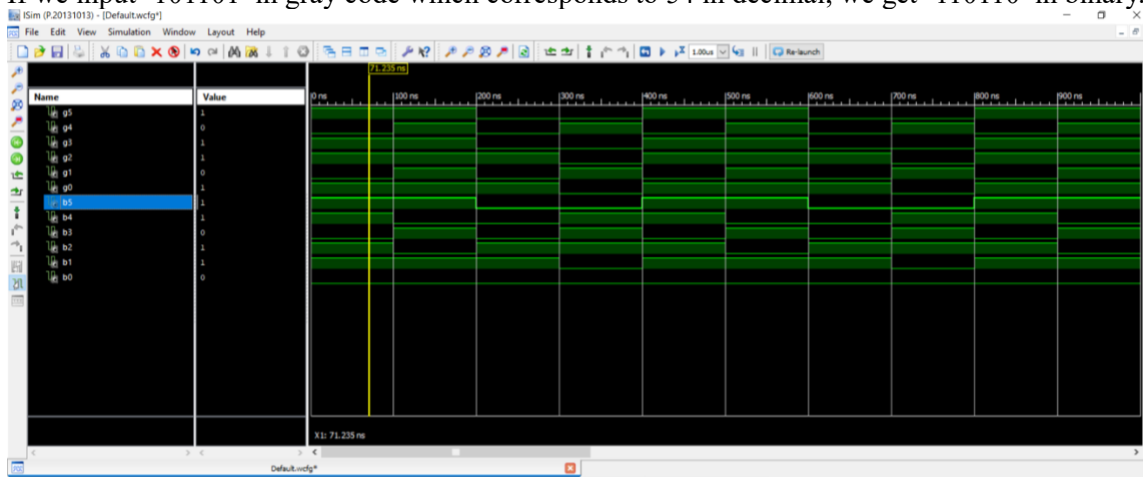
Algorithm behind this circuit is basically to first get MSB of the Gray code as MSB of binary (can easily be seen by looking at truth tables) and then XOR first bit with second bit and write the result as the second bit of binary. After that XOR the second bit of binary with third bit of Gray code and keep going until you reach the LSB.

For converting 6 bits I designed the circuit below. $g_5, g_4, \dots, g_0$ (g_5 being the most significant bit) are input of the gray code and $b_5, b_4, \dots, b_0$ (b_5 being the most significant bit) are outputs in binary.

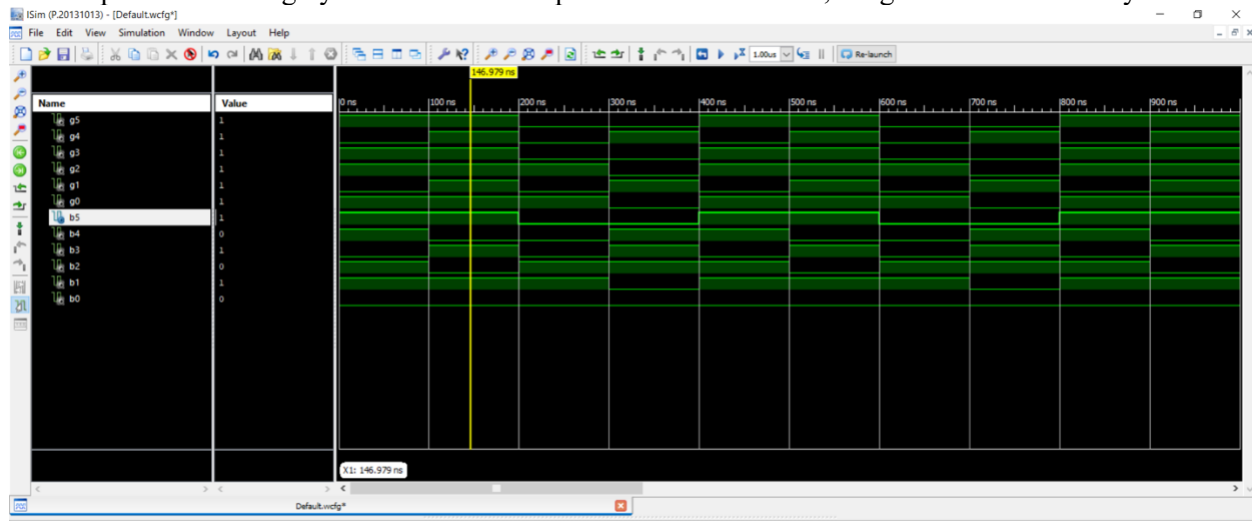


Note that a buffer must be placed between, if one wants to directly connect an input to an output. Otherwise, program will not be able to differentiate whether it's an input or output.

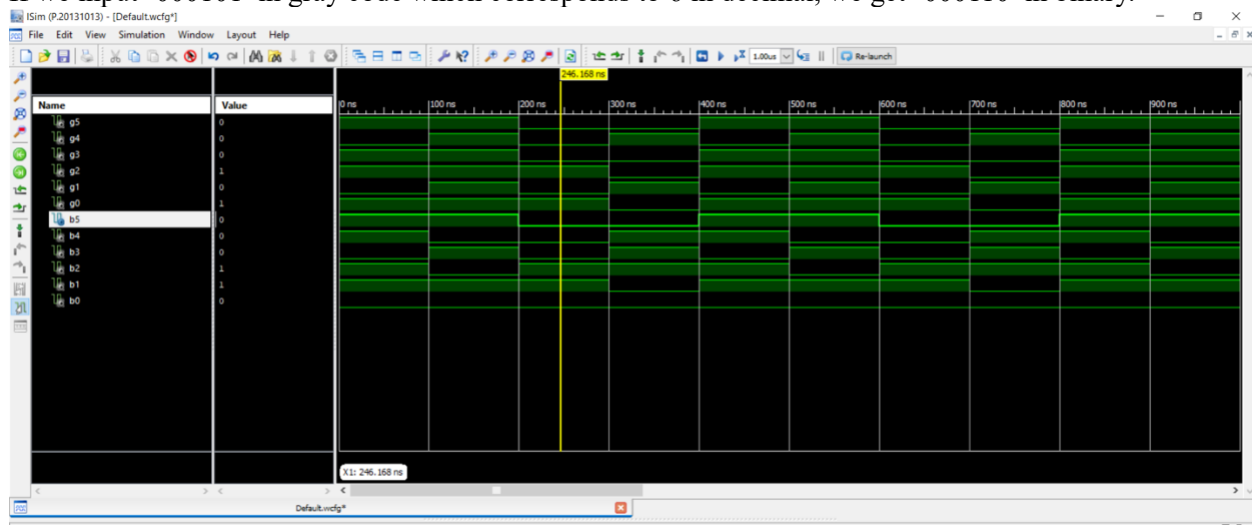
If we input '101101' in gray code which corresponds to 54 in decimal, we get '110110' in binary.



If we input '111111' in gray code which corresponds to 42 in decimal, we get '110110' in binary.



If we input '000101' in gray code which corresponds to 6 in decimal, we get '000110' in binary.



Part 2: Converting Binary to BCD

In this step we need to convert our 6-bit binary number to two 4-bit BCD digits (8-bits in total). During my initial thought process about this step, I couldn't come up with something much expect the idea that I somehow need to manipulate numbers between 10 to 15 (10 and 15 included) and get hold of a carry to add to the next digit because we can only represent 0 to 9 using BCD. After a quick search through the internet, I came across the double dabble algorithm.

“The “double dabble” algorithm is commonly used for converting a binary number to BCD. The binary number is left-shifted once for each of its bits, with bits shifted out of the MSB of the binary number and into the LSB of the accumulating BCD number. After every shift, all BCD digits are examined, and 3 is added to any BCD digit that is currently 5 or greater.

This works because every left shift multiplies all BCD digits by two. Since BCD digits cannot exceed nine, a pre-shift number of five or more would result in a post-shift number of ten or more, which cannot

be represented. Adding three to any BCD digit greater than five does two things: first, at the next shift, the 3 that was added becomes 6, and that accounts for the difference in binary and BCD codes (BCD uses 10 binary codes, and binary uses 16); and second, adding 3 forces the MSB of the BCD digit to a 1, where it is “carried out” and into the next digit. The figure below illustrates the process (the blue boxes around BCD digits show BCD digits that are ≥ 5 , and therefore need 3 to be added).” [3]

BCD2	BCD1	BCD0	Binary Input	
0000	0000	0000	10010111	
0000	0000	0001	0010111.	← ①
0000	0000	0010	010111..	← ②
0000	0000	0100	10111...	← ③
0000	0000	1001	0111....	← ④
0000	0000	1100	0111....	ADD
0000	0001	1000	111....	← ⑤
0000	0001	1011	111....	ADD
0000	0011	0111	11....	← ⑥
0000	0011	1010	11....	ADD
0000	0111	0101	1....	← ⑦
0000	1010	1000	1....	ADD ADD
0001	0101	0001		← ⑧

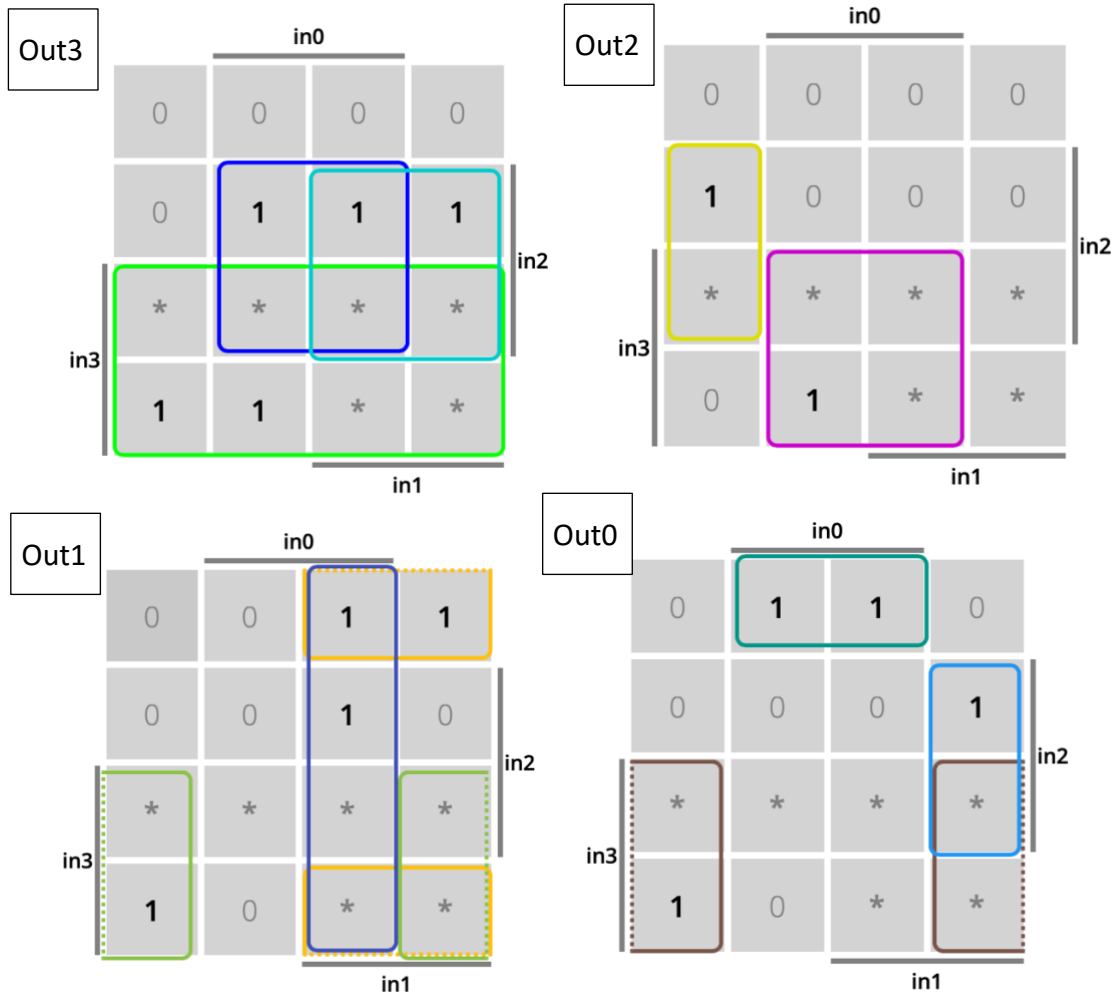
[3]

So we need a combinational circuit element that will give the same number if it’s smaller than 5 and add 3 if it’s equal or greater than 5. Let’s call this element “conditional adder”. To construct our conditional adder we first need to draw a truth table.

in3	in2	in1	in0	out3	out2	out1	out0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	0
0	0	1	1	0	0	1	1
0	1	0	0	0	1	0	0
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0	x	x	x	x
1	0	1	1	x	x	x	x
1	1	0	0	x	x	x	x
1	1	0	1	x	x	x	x
1	1	1	0	x	x	x	x
1	1	1	1	x	x	x	x

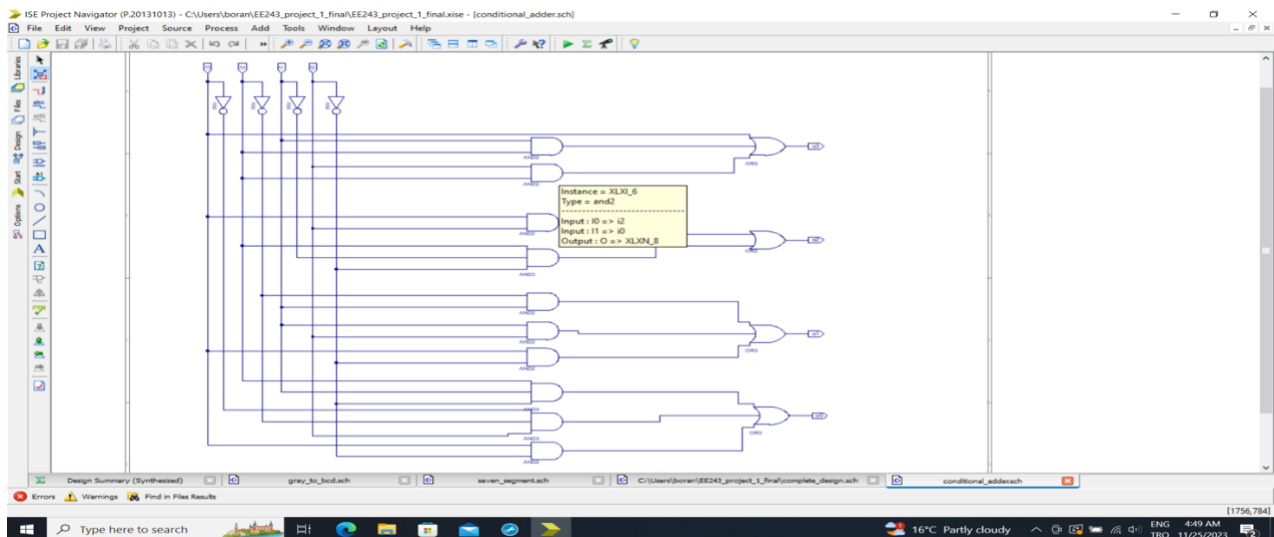
[4]

We put don’t care for 10 through 15 because conditional adder will never take those values as input. After that we can draw K-maps for each output bit:

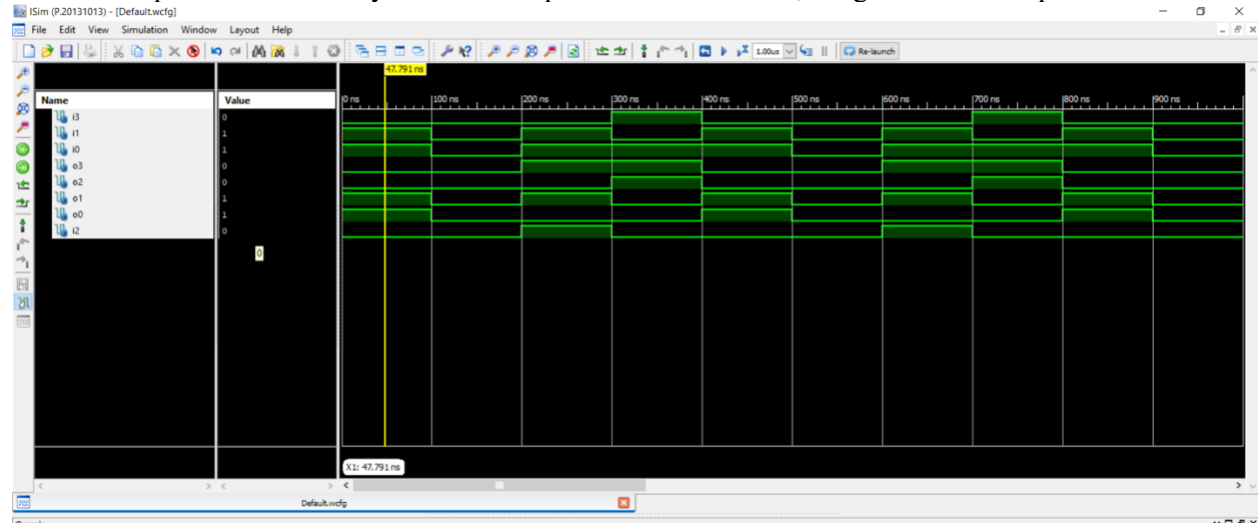


[5]

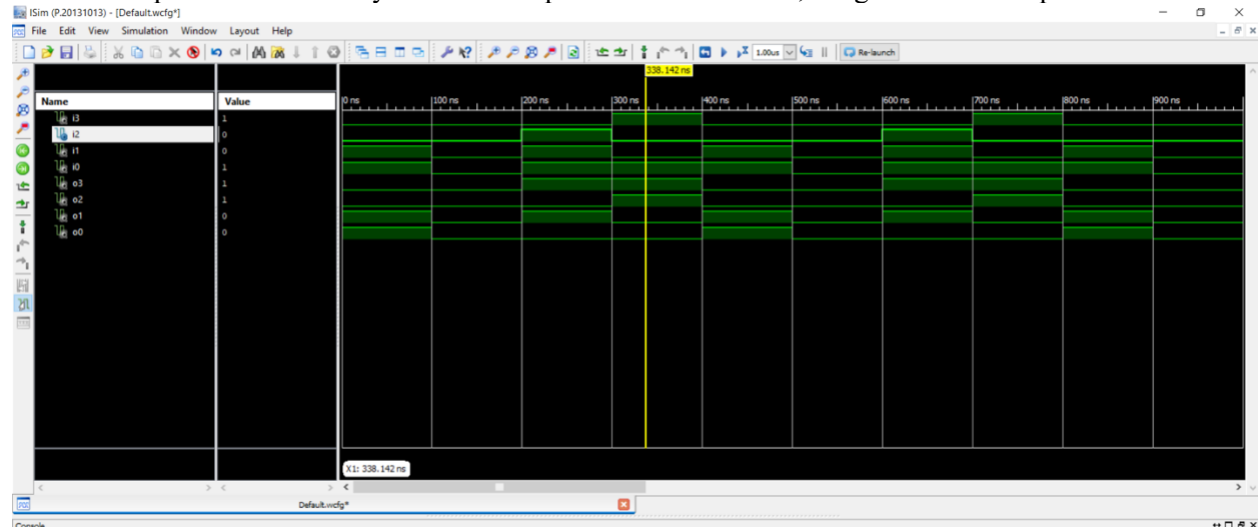
And using K-maps I constructed the conditional adder in Xilinx. i3, i2, i1, i0 (i3 being the most significant bit) are the inputs and o3, o2, o1, o0 (o3 being the most significant bit) are the outputs.



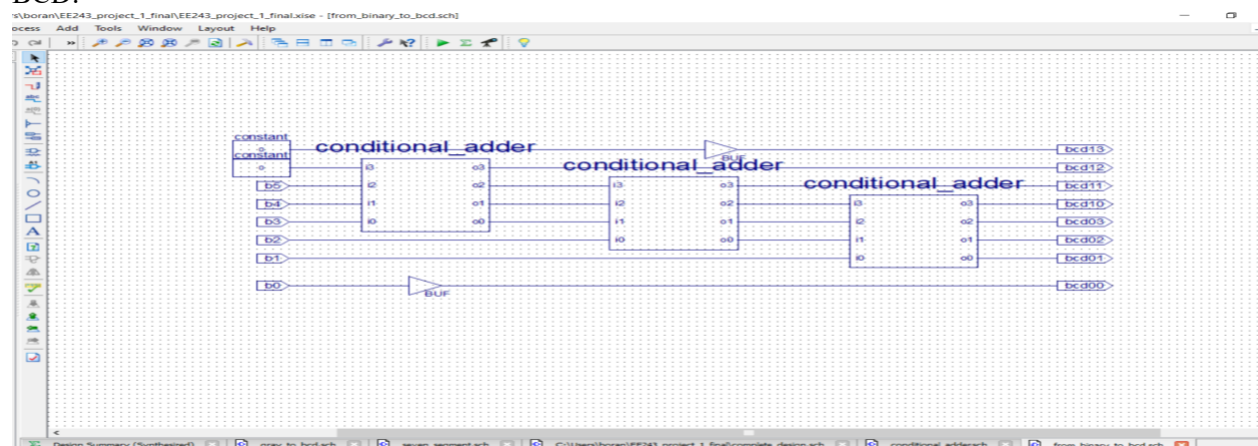
When we input '0011' in binary which corresponds to 3 in decimal, we get '0011' as expected.



When we input '1001' in binary which corresponds to 9 in decimal, we get '1100' as expected.



Now using conditional adders as our building blocks, we can construct the circuit that converts binary to BCD.

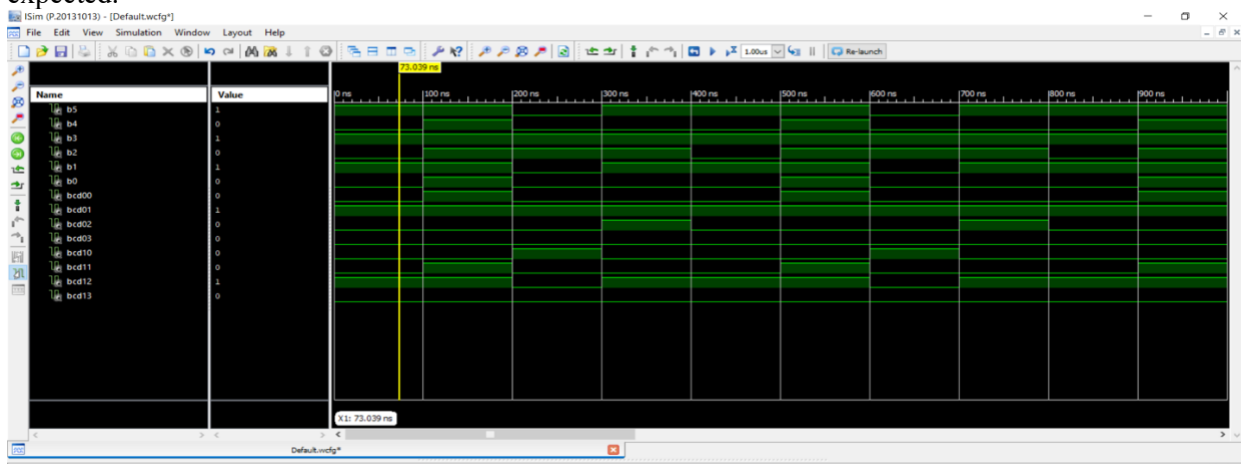


b5, b4, ..., b0 (b5 being the most significant bit) are inputs in binary; b03, b02, b01, b00 (b03 being the most significant bit) are 4 bits of the first digit (one's digit) and b13, b12, b11, b10 (b13 being the most significant bit) are 4 bits of the second digit (ten's digit) of the BCD number.

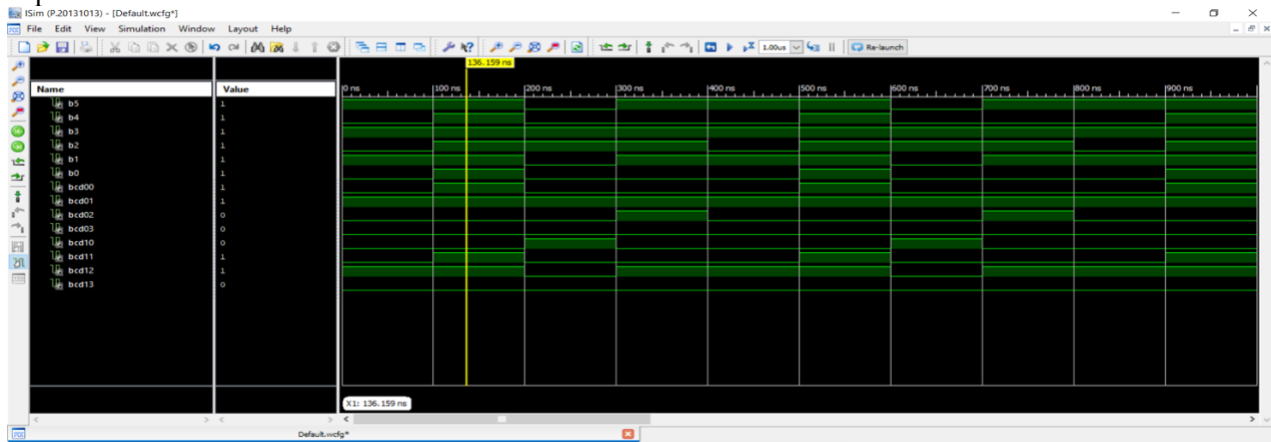
Observe that b13 is always 0 because we can't obtain any number larger than 63 using a 6-bit binary number. Also b00 is always equal to b0 because b0 determines whether the number is even or odd. If b0=0 number is even and thus least significant bit of one's digit must also be equal to 0. Same also goes for b0=1.

The reason we shift our conditional adders after each iteration is shifting by 1 bit to the left corresponds to multiplying the binary number by 2. I have used this property to implement the 'double dabble' algorithm in my design.

When we input '101010' in binary which corresponds to 42 in decimal, we get '0100'- '0010' in BCD as expected.

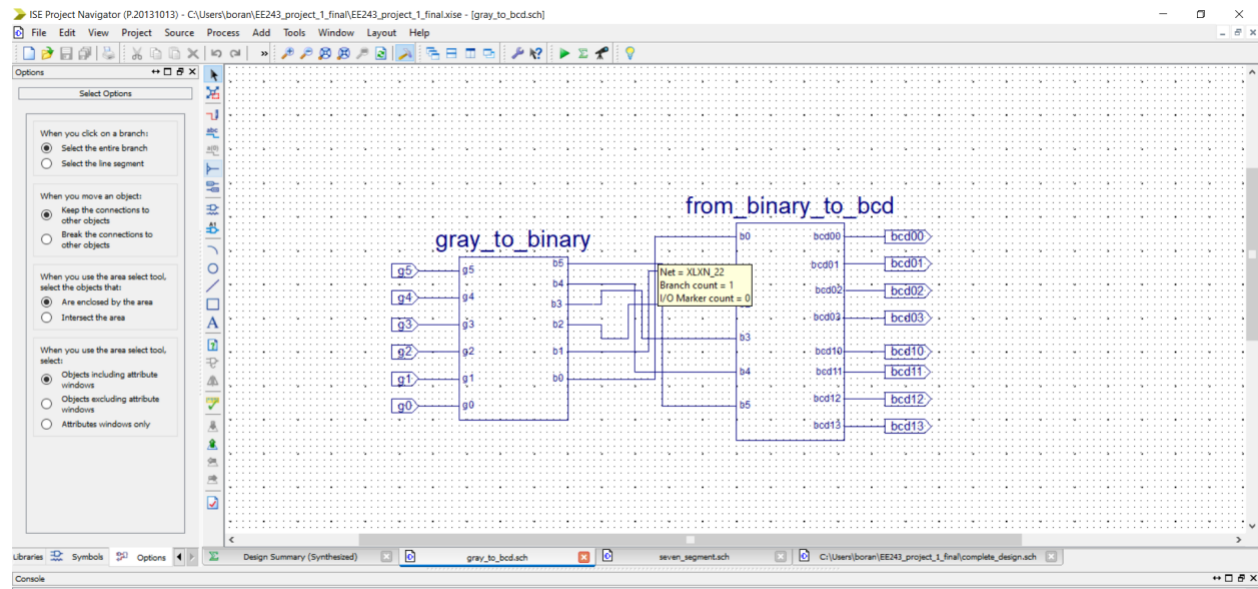


When we input '111111' in binary which corresponds to 63 in decimal, we get '0110'- '0011' in BCD as expected.

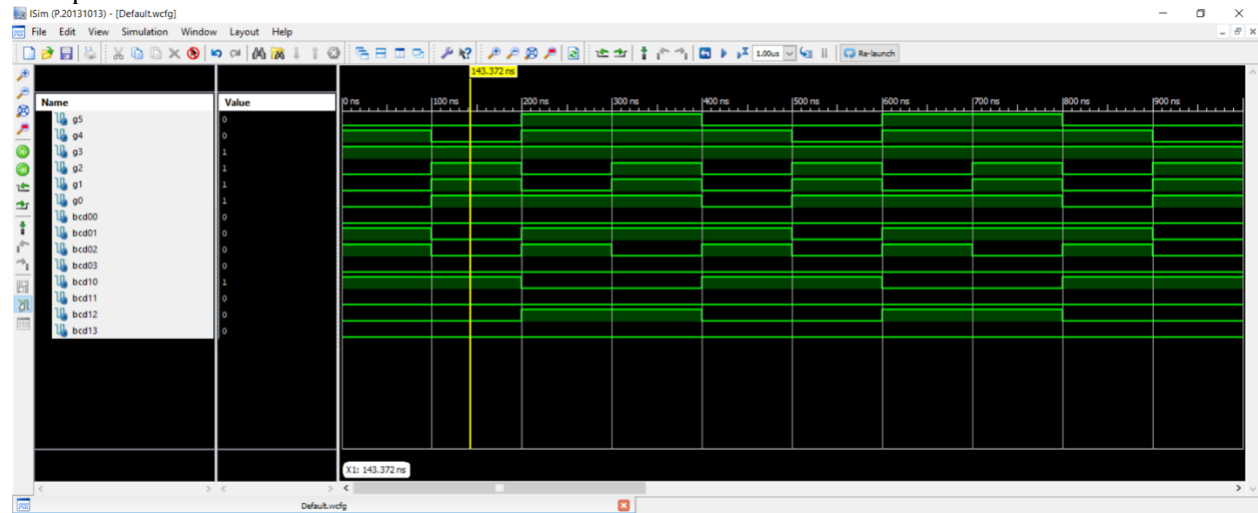


Since we have constructed a circuit that converts gray code to binary and a one that converts binary to BCD. We can combine these to obtain a circuit that converts gray code to BCD. We can create schematic symbols of our circuits in Xilinx for convenience.

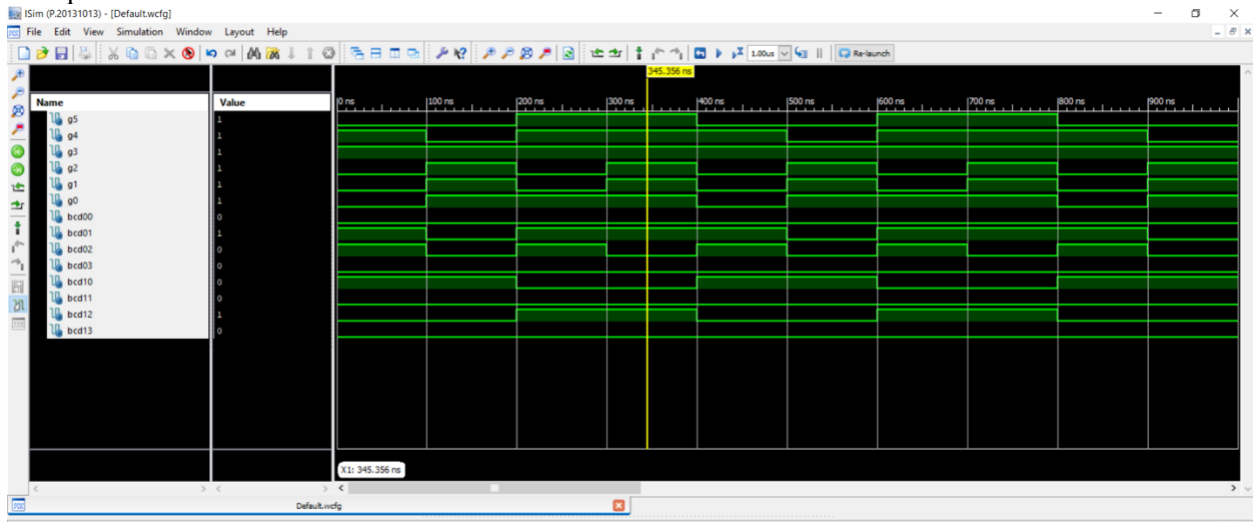
g5, g4,...g0 are the inputs in gray code; b03, b02, b01, b00 (b03 being the most significant bit) are 4 bits of the first digit (one's digit) and b13, b12, b11, b10 (b13 being the most significant bit) are 4 bits of the second digit (ten's digit) of the BCD number.



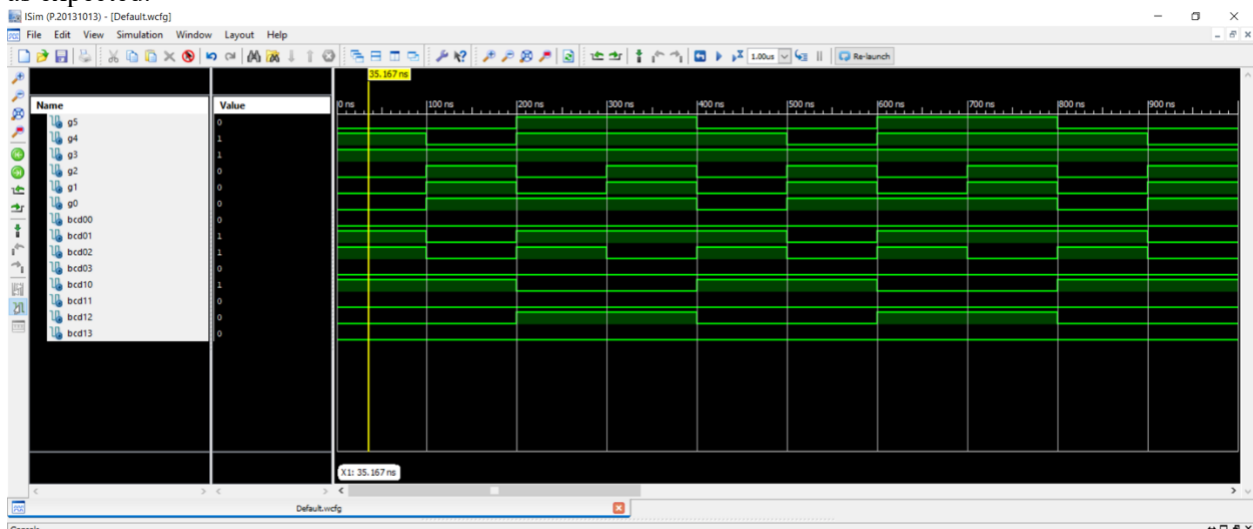
When we input '001111' in gray code which corresponds to 10 in decimal, we get '0001'-'0000' in BCD as expected.



When we input '111111' in gray code which corresponds to 42 in decimal, we get '0100'-'0010' in BCD as expected.



When we input '011000' in gray code which corresponds to 16 in decimal, we get '0001'-'0110' in BCD as expected.



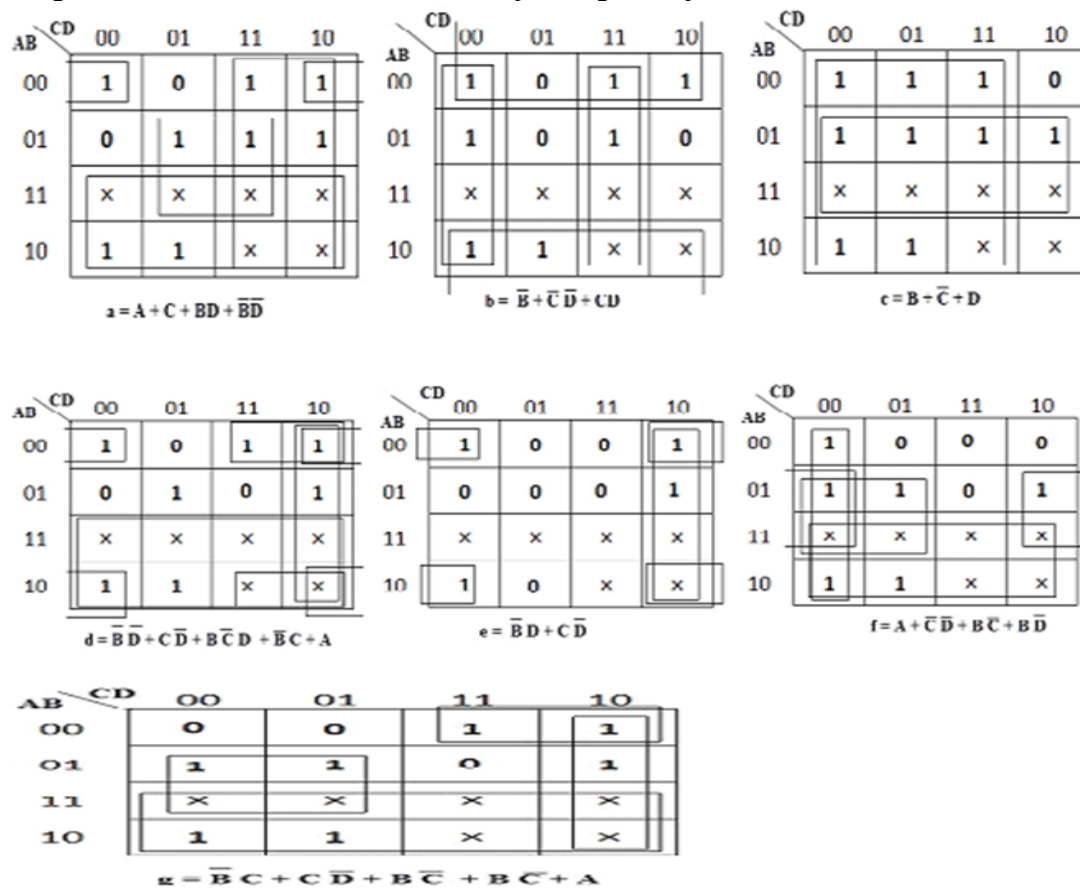
Part 3: Representing BCD Digits with 7 Segment Display

There is no direct logical relationship between a binary number (BCD is same with binary up to 9) and their seven-segment representation because which segments of the display will light up when the corresponding bits are given has nothing to do with the binary system or any logical order. Which LED's will light up is completely up to how a number is physically written. For example, if we choose to represent number six with something like "|-|" our circuit would change although the value of six stays the same. So, in order to explore which combinations of 4 bits would cause a particular LED to light up, we need 4 variable Karnaugh maps for each of the 7 segments.

Decimal Digit	Input lines				Output lines							Display pattern
	A	B	C	D	a	b	c	d	e	f	g	
0	0	0	0	0	1	1	1	1	1	1	0	0
1	0	0	0	1	0	1	1	0	0	0	0	1
2	0	0	1	0	1	1	0	1	1	0	1	2
3	0	0	1	1	1	1	1	1	0	0	1	3
4	0	1	0	0	0	1	1	0	0	1	1	4
5	0	1	0	1	1	0	1	1	0	1	1	5
6	0	1	1	0	1	0	1	1	1	1	1	6
7	0	1	1	1	1	1	1	0	0	0	0	7
8	1	0	0	0	1	1	1	1	1	1	1	8
9	1	0	0	1	1	1	1	1	0	1	1	9

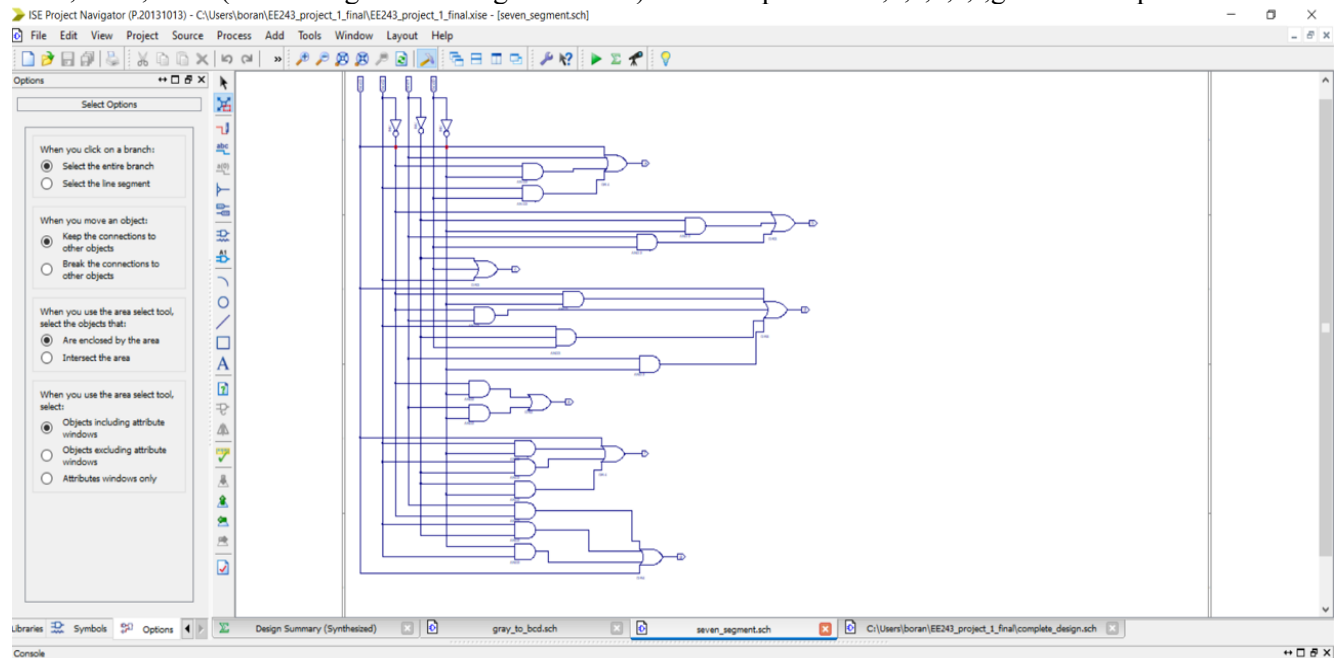
[6]

Notice that we don't care for numbers 10 through 15 because they are not represented in BCD. Using the truth table we can draw the corresponding K-maps as follows.

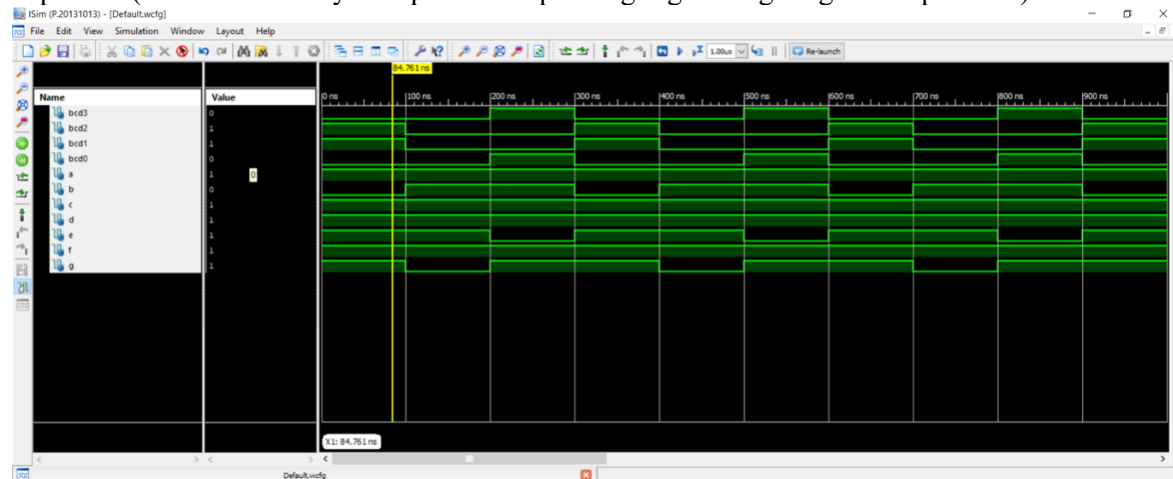


[7]

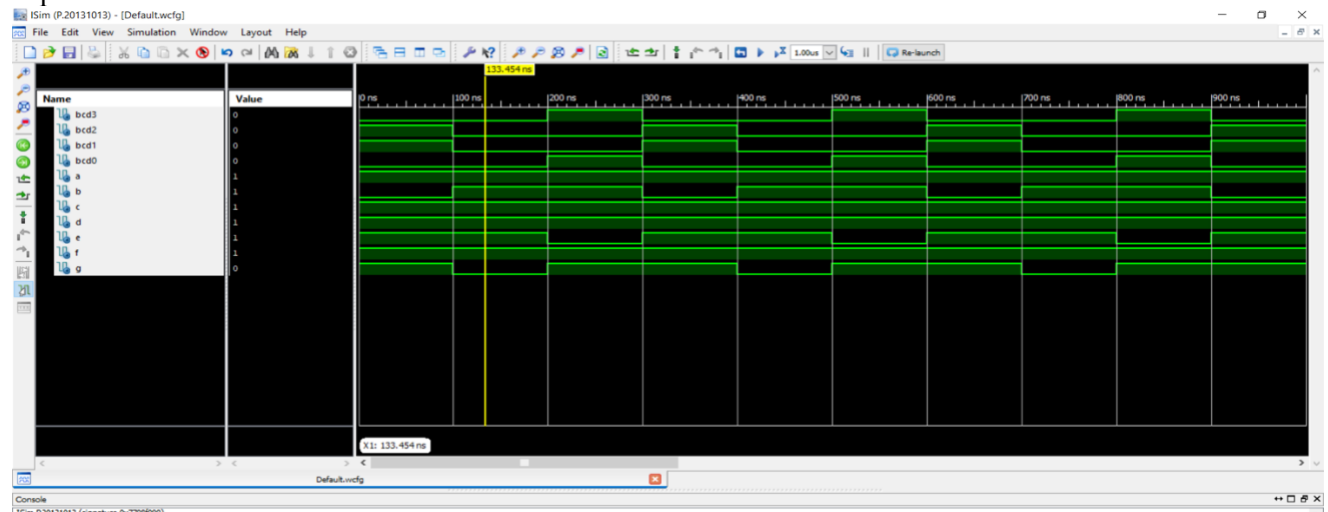
By utilizing K-maps we can construct the circuit that converts BCD to 7 segment display. Here bcd3, bcd2, bcd1, bcd0 (bcd3 being the most significant bit) are the inputs and a,b,c,d,e,f,g are the outputs.



When we input '0110' in BCD which corresponds to 6 in decimal, segments 'a,c,d,e,f,g' are lit up as expected. (What is meant by 'lit up' is corresponding segments giving the output of 1.)

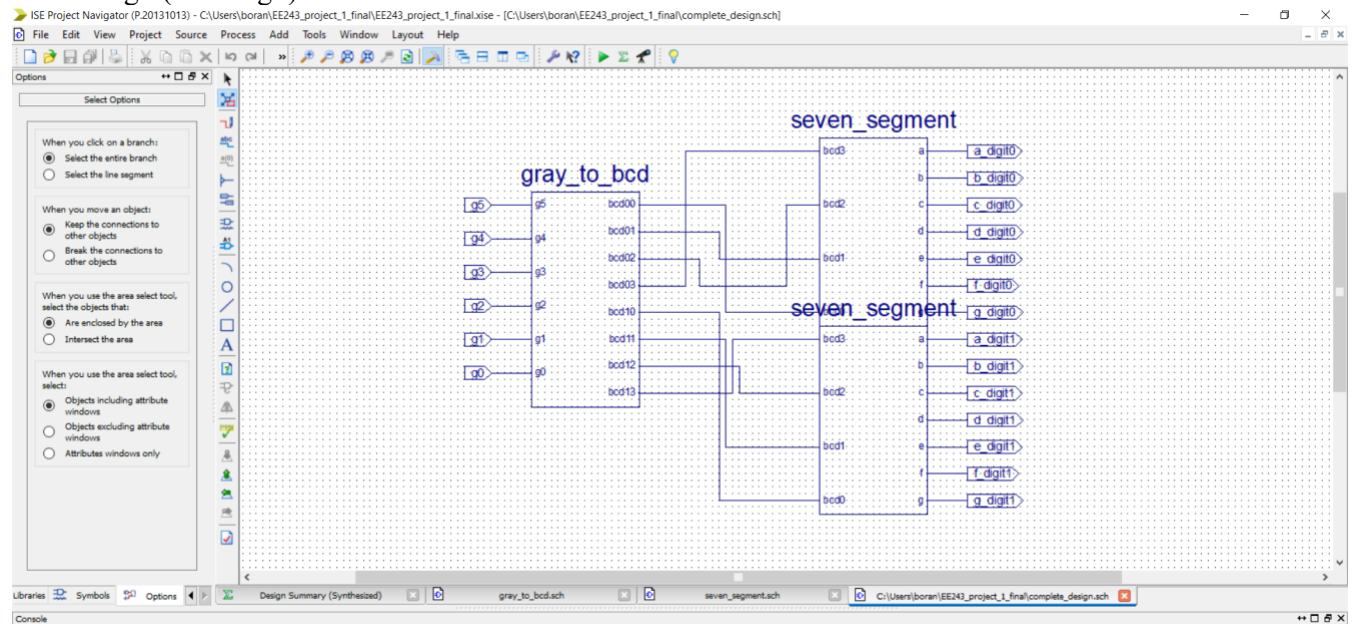


When we input '0000' in BCD which corresponds to 0 in decimal, segments 'a,b,c,d,e,f' are lit up as expected.

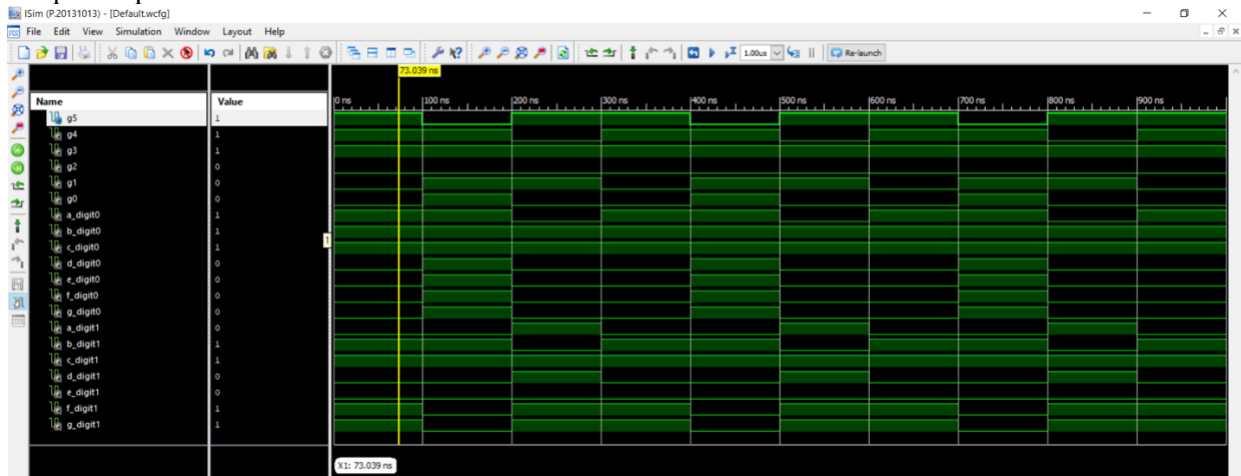


Now having constructed a circuit that converts gray code to BCD and another circuit that converts BCD to 7-segment display, we can put it all together.

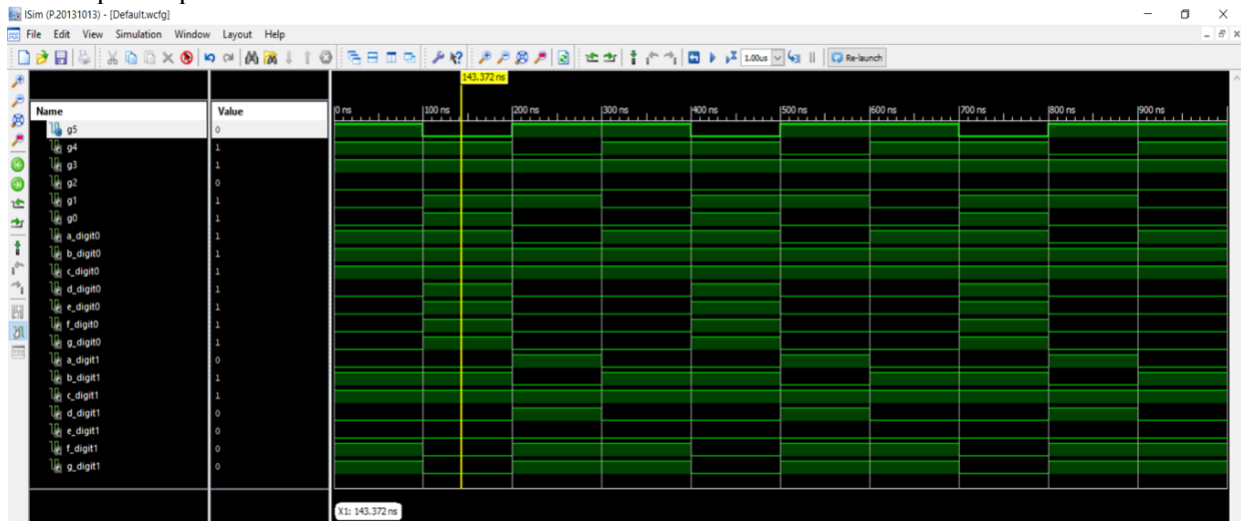
Here g5, g4,..g0 (g5 being the most significant bit) are inputs in gray code; a_digit0,b_digit0,...,e_digit0 are 7 segments of the first digit (one's digit) and a_digit1,b_digit1,...,e_digit1 are 7 segments of the second digit (ten's digit).



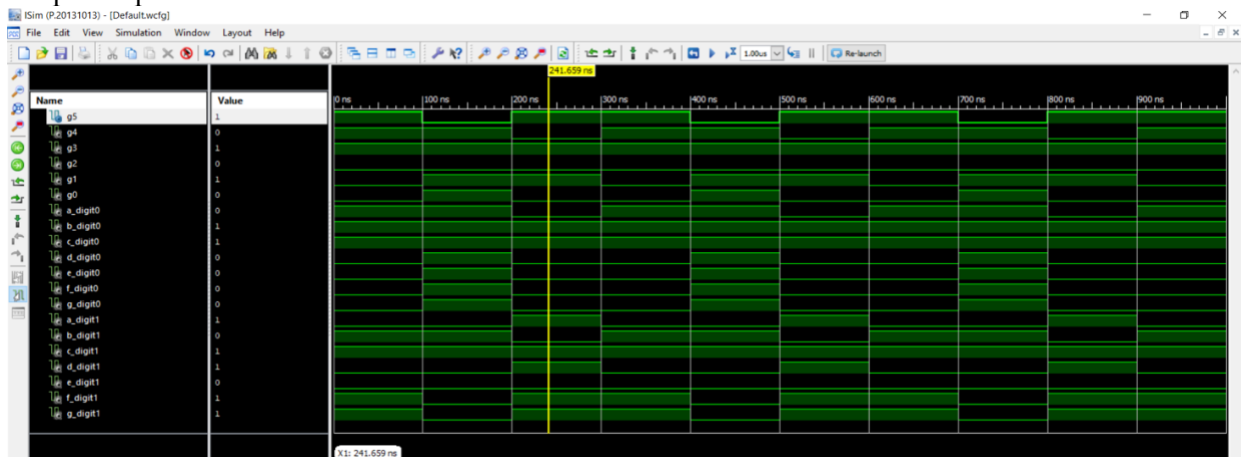
When we input '111000' in gray code which corresponds to 47 in decimal, segments 'b,c,f,g'-'a,b,c' are lit up as expected.



When we input '011011' in gray code which corresponds to 18 in decimal, segments 'b,c'-'a,b,c,d,e,f,g' are lit up as expected.



When we input '101010' in gray code which corresponds to 51 in decimal, segments 'a,c,d,f,g'-'b,c' are lit up as expected.



References:

- [1]: <https://sebastian.itch.io/digital-logic-sim>
- [2]: Digital Design – Solution Manual. M. Mano. M.D. Ciletti, pp: 57
- [3]: <https://www.realdigital.org/doc/6dae6583570fd816d1d675b93578203d>
- [4]: <https://electronics.stackexchange.com/questions/406580/how-do-i-make-a-double-dabble-circuit-with-logic-gates>
- [5]: <https://tams.informatik.uni-hamburg.de/research/software/tams-tools/kvd-editor.html> is used for drawing K-maps
- [6]: <https://www.electrical4u.com/bcd-to-seven-segment-decoder/>
- [7]: https://www.researchgate.net/figure/Figure-4-Karnoughs-Map-simplification-of-the-7-Segment-Decoder_fig3_340621221